AI Hallucination Detection & Verification System

Complete Technical Stack, Configuration Specification & System Architecture Guide

Author / System: Anti-Hallucination Engineering Platform | **Frameworks:** FastAPI, Python 3.11+, Pydantic v2, SQLite | **Status:** Production Ready (Local & Cloud)

This document serves as the official technical manual for the **AI Hallucination Detection and Verification System**. Rather than relying on superficial keyword matching or vulnerable single-prompt self-consistency loops, the platform breaks AI responses into atomic, typed propositions, conducts independent evidence retrieval, scores domain credibility, detects fine-grained contradictions (temporal, numerical, entity), subjects verdicts to an adversarial critic, and computes deterministic reliability metrics.

1. System Architecture & Multi-Agent Hierarchy

The verification pipeline executes asynchronously across 6 specialized agents:

```
USER INPUT (Question or Pasted Answer)
   │
   ▼
AGENT 1: ANSWER GENERATOR (LLM Generation or Passthrough User Text)
   │
   ▼
AGENT 2: ATOMIC CLAIM EXTRACTOR (Syntactic & Semantic Decomposition into 12+ Typed Claims)
   │
   ▼
CLAIM NORMALIZER & QUERY PLANNER (Targeted Search Queries & Entity Anchoring)
   │
   ▼
AGENT 3: EVIDENCE RETRIEVER (Wikipedia REST API + DuckDuckGo Web Search + HTML Chunk Scraper)
   │
   ▼
SOURCE QUALITY EVALUATOR (Explainable 0–100 Rubric: Provenance, Recency, Semantic Alignment)
   │
   ▼
AGENT 4: ZERO-TRUST VERIFIER (Temporal, Numerical & Inter-Source NLI Verification)
   │
   ▼
AGENT 5: ADVERSARIAL CRITIC (Catches Confirmation Bias, Weak Provenance & Uncited Claims)
   │
   ▼
AGENT 6: HALLUCINATION SCORING ENGINE (Harmonic Reliability 0–100 & Hallucination Risk %)
   │
   ▼
SQLITE AUDIT PERSISTENCE & INTERACTIVE REAL-TIME DASHBOARD UI
```

> **Zero-Trust Verification Principle**
> An absence of evidence strictly yields `UNVERIFIED` —it is never guessed as false or assumed true. Claims are grounded exclusively in retrieved evidence tokens with verifiable citations.

2. Comprehensive Technology Stack

Layer / Subsystem	Technology & Version	Role & Technical Responsibility
Core Backend	`FastAPI` (v0.143.0) `Uvicorn` (v0.54.0) `Starlette` (v1.7.0)	High-concurrency ASGI REST API framework with asynchronous request handlers, CORS middleware, lifespan startup events, and interactive Swagger documentation at `/docs`.
Data Modeling & Validation	`Pydantic v2` (v2.14.1) `pydantic-settings` (v2.15.0) `pydantic-core` (v2.50.1)	Strict type validation, structured domain models (`Claim` , `EvidenceItem` , `VerificationResult` , `VerificationReport`), and environment parsing with fallbacks.
HTTP & Web Scraping	`httpx` (v0.28.1) `BeautifulSoup4` (v4.15.0) `duckduckgo-search` (v8.1.1) `primp` (v2.0.1) / `lxml` (v6.1.3)	Async HTTP client with custom timeouts and redirect tracing; DuckDuckGo text search; Wikipedia Action API querying; HTML tag stripping and paragraph-level chunk extraction via `PageScraper` .
Database & Audit Store	`SQLite 3` (Native) Python `sqlite3` `contextlib` Connection Pooling	Embedded relational audit repository. Manages normalized schemas across `reports` , `claims` , and `evidence` tables with Foreign Keys and `ON DELETE CASCADE` constraints.
AI & Verification Engine	Dual-Layer Hybrid: 1. Rule-based NLI heuristics 2. Live LLM API integration	Deterministic offline heuristics for dates, numbers, and relationships; optional live frontier model calls via `google-genai` (Gemini 2.5 Flash), OpenAI (GPT-4o-mini), or Groq.
Testing & Benchmark Harness	`pytest` (v9.1.1) `pytest-asyncio` (v1.4.0) Custom Adversarial Harness	Automated test suite across agents, retrieval modules, scoring formulas, and 10 adversarial benchmarks measuring Precision, Recall, and F1.
Frontend User Interface	Semantic `HTML5` Modern `Vanilla CSS3` `Vanilla JavaScript (ES6+)`	Zero-dependency, high-contrast dashboard with live pipeline progress stepper, color-coded verdict badges, evidence preview drawer, and JSON/text export facilities.
Process & Dev Tooling	Python `venv` `package.json` (NPM proxy) `start_app.bat` (Windows Launcher)	Cross-tool developer workflow: standard CLI (`python run.py`), Windows batch script, and NPM scripts (`npm run dev` , `npm test` , `npm run benchmark`).

2.1 Frontend UI Features & Design Tokens

- **Glassmorphism & Color Variables:** Modern CSS tokens for dark/light themes, subtle gradients, and card shadows.
- **Responsive Stepper:** Real-time visual progress indicator tracking *Generation → Extraction → Normalization → Retrieval → Verification → Audit Critic → Scoring*.
- **Verdict Badges:** High-contrast badges for `SUPPORTED` , `PARTIALLY_SUPPORTED` , `CONTRADICTED` , `UNVERIFIED` , and `NOT_FACTUAL` .
- **Audit Export:** Interactive copy and download buttons for raw JSON reports and formatted plain-text forensic logs.

3. System Configurations Reference

Configurations are loaded dynamically from environment variables and an optional `.env` file via `backend/app/config.py`.

Variable Name	Default Value	Type	Description & Impact
`HOST`	`0.0.0.0`	String	Network interface IP binding for the Uvicorn web server. Set to `127.0.0.1` for local-only access.
`PORT`	`8000`	Integer	TCP port on which the FastAPI application listens for incoming HTTP requests.
`DEBUG`	`true`	Boolean	Enables development mode, hot-reloading (`StatReload`), and verbose logging output.
`DATABASE_PATH`	`./hallucination_verification.db`	Path	Filesystem location of the SQLite audit database file.
`DATABASE_URL`	`sqlite:///./hallucination_verification.db`	String	Connection string URI formatted for SQLite database connectivity.
`DEFAULT_MODEL`	`hybrid-verifier-v1`	String	Model tag used when generating answers. Options: `hybrid-verifier-v1` , `gemini-2.5-flash` , `gpt-4o-mini` .
`MAX_SEARCH_RESULTS`	`5`	Integer	Maximum number of evidence snippets retrieved per search query from DuckDuckGo and Wikipedia.
`REQUEST_TIMEOUT_SECONDS`	`15`	Integer	Global HTTP request timeout in seconds applied to external search APIs and web scraping requests.
`ENABLE_CRITIC_PASS`	`true`	Boolean	Enables Agent 5 (Adversarial Critic) to audit and overturn lax verdicts before calculating reliability scores.
`EVIDENCE_CONFIDENCE_THRESHOLD`	`0.65`	Float	Minimum semantic alignment score required before external evidence is admitted for claim verification.
`GEMINI_API_KEY` / `GOOGLE_API_KEY`	*(empty)*	String	Optional Google AI Studio API key for enabling live Gemini LLM answer generation and semantic NLI verification.
`OPENAI_API_KEY`	*(empty)*	String	Optional OpenAI API key for GPT-4o-mini fallback verification.
`GROQ_API_KEY`	*(empty)*	String	Optional Groq API key for low-latency Llama-3 inference.

> **Offline & Zero-Cost Operation**
> When all LLM API keys are unset, the system functions seamlessly in **hybrid heuristic mode**. It utilizes the Wikipedia REST API and DuckDuckGo search without incurring any third-party API fees.

4. Scoring Formulas, Rubric & Database Schema

4.1 Source Quality Scoring Rubric (0–100)

Source Category	Base Score	Domain Patterns & Qualification Criteria
Government & Official	92.0	`.gov`, `.mil`, `.gov.uk`, `.europa.eu`, `who.int`, `nasa.gov`. Primary official provenance.
Academic & Peer-Reviewed	88.0	`.edu`, `arxiv.org`, `nature.com`, `science.org`, `ncbi.nlm.nih.gov`, `ieee.org`.
Tier-1 Journalism	80.0	`reuters.com`, `apnews.com`, `bbc.com`, `nytimes.com`, `wsj.com`, `theguardian.com`.
Curated References	75.0	`wikipedia.org`, `britannica.com`, `plato.stanford.edu`, `worldcat.org`.
General Web Public	55.0	Standard verified blogs, corporate publications, documentation portals.
User-Generated Content	25.0	`reddit.com`, `quora.com`, `twitter.com`, `x.com`, `medium.com`, `wordpress.com`.

Relevance Adjustment: `Score = BaseScore + (Relevance - 0.5) * 15.0` (clamped to [5.0, 99.0]).

4.2 Reliability & Risk Formulation

Overall Reliability (0–100)

Weighted harmonic aggregation across factual claims:

`Reliability = (∑ W_i × VerdictScore_i) / (∑ W_i) × 100`

- `W_i = Importance × Confidence`
- Supported: `1.0 × min(1.0, max(0.6, SQ / 85.0))`
- Partially Supported: `0.65 × min(1.0, max(0.6, SQ / 85.0))`
- Unverified: `0.35` (uncertainty floor) | Contradicted: `0.00`

Hallucination Risk (%)

Quantifies verified errors & epistemic uncertainty:

`Risk = (∑ W_i × RiskFactor_i) / (∑ W_i) × 100`

- Contradicted: `1.00` (confirmed hallucination)
- Unverified: `0.50` (epistemic uncertainty penalty)
- Partially Supported: `0.35` (minor factual deviation)
- Supported: `0.00`

4.3 Database Schema (SQLite)

```
CREATE TABLE reports (
    report_id TEXT PRIMARY KEY, question TEXT NOT NULL, answer TEXT NOT NULL,
    model_name TEXT, created_at TEXT NOT NULL, overall_reliability REAL,
    hallucination_risk REAL, summary_counts TEXT, score_breakdown TEXT, explanation TEXT, raw_json TEXT
);
CREATE TABLE claims (
    claim_id TEXT NOT NULL, report_id TEXT NOT NULL, claim_text TEXT NOT NULL,
    claim_type TEXT NOT NULL, importance REAL, requires_verification INTEGER,
    verdict TEXT, confidence REAL, PRIMARY KEY (report_id, claim_id),
    FOREIGN KEY (report_id) REFERENCES reports (report_id) ON DELETE CASCADE
);
CREATE TABLE evidence (
    evidence_id TEXT NOT NULL, report_id TEXT NOT NULL, claim_id TEXT NOT NULL,
    source_title TEXT, source_url TEXT, source_domain TEXT, source_type TEXT,
    snippet TEXT, relevance REAL, source_quality_score REAL, PRIMARY KEY (report_id, evidence_id),
    FOREIGN KEY (report_id) REFERENCES reports (report_id) ON DELETE CASCADE
);
```

5. REST API & Operational Execution Matrix

5.1 API Endpoints Specification

Method	Endpoint	Payload / Params	Response Structure
GET	`/api/health`	None	`{ status, service, version, default_model, database, critic_enabled }`
POST	`/api/generate`	`{ question, model_name? }`	`{ answer, model, question }`
POST	`/api/claims`	`{ text, question? }`	`[ { claim_id, claim, claim_type, importance, requires_verification } ]`
POST	`/api/verify`	`{ claim, evidence_list }`	`{ claim_id, verdict, confidence, reason, evidence_ids }`
POST	`/api/analyze`	`{ question, answer?, model_name?, enable_critic? }`	Full `VerificationReport` (claims, verdicts, evidence citations, reliability, risk)
GET	`/api/reports`	`?limit=20`	List of recent verification report summaries
GET	`/api/reports/{id}`	Path: `report_id`	Complete structured report in JSON format
GET	`/api/reports/{id}/text`	Path: `report_id`	Formatted plain-text forensic audit report

5.2 Operational Commands Matrix

Operational Task	Command (CLI / NPM / Batch)	Execution Description
Start Dev Server	`npm run dev` or `python run.py`	Launches FastAPI + Uvicorn on `http://127.0.0.1:8000` with auto-reload.
Windows One-Click	`start_app.bat`	Launches server using `.venv\Scripts\python.exe` in a dedicated command window.
Run PyTest Suite	`npm test` or `python run.py --test`	Executes agent, retriever, verifier, and scoring test suite.
Adversarial Benchmark	`npm run benchmark` or `python run.py --benchmark`	Runs 10 adversarial benchmarks, calculating Precision, Recall, F1, and Latency.